

If Else

Topics covered: if, if-else, else if ladder, nested if-else, switch statement, logical operator combinations, short-circuit evaluation, common condition-writing mistakes.

Part 1 — Notes

1. Basic if

```
if (condition) {  
    // runs only if condition is true  
}
```

Condition must evaluate to boolean — Java does not allow int as a condition (unlike C/C++).

2. if-else

```
if (condition) {  
    // true block  
} else {  
    // false block  
}
```

3. else if Ladder

```
if (marks >= 90) {  
    grade = "A";  
} else if (marks >= 75) {  
    grade = "B";  
} else if (marks >= 50) {  
    grade = "C";  
} else {  
    grade = "F";  
}
```

Order matters: conditions are checked top to bottom, and the first true one runs — rest are skipped, even if they'd also be true.

Common mistake: writing conditions in the wrong order, e.g. checking marks >= 50 before marks >= 90 — then a 95 would wrongly get grade C, since the ladder stops at the first match.

4. Nested if-else

```
if (a > b) {  
    if (a > c) {  
        // a is largest  
    } else {  
        // c is largest  
    }  
} else {
```

```
// b or c is largest – needs another check
}
```

Used when a decision depends on another decision — e.g. checking a triangle is valid, AND THEN checking what type it is.

5. switch Statement

```
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Invalid");
}
```

Use switch instead of a long else-if ladder when checking ONE variable against multiple exact values (not ranges). Common mistake: forgetting break — execution 'falls through' into the next case even if it doesn't match.

6. Logical Operator Combinations in Conditions

```
if (age >= 18 && hasID) { } // both must be true
if (isWeekend || isHoliday) { } // either can be true
if (!isBlocked) { } // true when isBlocked is false
```

Short-circuit evaluation: in a `&& b`, if 'a' is false, Java never checks 'b' — it already knows the result is false. Same for `||` — if 'a' is true, 'b' is skipped.

This matters for code like: `if (x != 0 && 10/x > 2)` — safe because if x is 0, the division is never reached. Reversing the order would crash on division by zero.

7. Comparing Values Correctly

Common mistake: using `==` to compare Strings. Use `.equals()` instead — `==` checks memory reference for objects, not actual content.

```
String s1 = "hi"; if (s1.equals("hi")) { } // correct
if (s1 == "hi") { } // works sometimes by luck, unreliable – avoid
```

For primitive types (int, char, double, boolean) `==` is correct and safe to use.

8. Ternary as a Condition Shortcut

```
String result = (marks >= 40) ? "Pass" : "Fail"; // one-line if-else for simple cases
```

Quick Revision Checklist

- Can I explain why else-if order matters, with an example where wrong order breaks logic?
- Do I know when to use switch instead of else-if (exact value match vs range check)?
- Can I explain short-circuit evaluation and why it matters (e.g. avoiding divide-by-zero)?

- Do I know why `==` fails for String comparison, and why `.equals()` is correct?
- Can I write a nested if-else for a 'decision depends on another decision' scenario from scratch?
- Do I remember that forgetting `break` in switch causes fall-through?

Part 2 — Practice Questions

Covers every subtopic above: basic if-else, else-if ladder ordering, nested conditions, switch, short-circuit logic, and String comparison — so no variety is missed.

Easy (5)

- 1 Take a number as input and print whether it's positive, negative, or zero.
- 2 Take three numbers and print the largest one using if-else (not ternary).
- 3 Check if a given year is a leap year.
- 4 Take a character input and check if it's a vowel or consonant.
- 5 Take marks as input and print the grade (A/B/C/F) using an else-if ladder — write the conditions in the correct top-to-bottom order.

Medium (5)

- 1 Take three sides of a triangle and check if they form a valid triangle, and if so, whether it's equilateral, isosceles, or scalene (uses nested if-else).
- 2 Take a day number (1-7) as input and print the day name using switch — include a default case for invalid input.
- 3 Take a number and a divisor as input; using short-circuit logic (`divisor != 0 && ...`), safely check and print whether the number is divisible by the divisor without crashing on `divisor = 0`.
- 4 Take two String inputs and check if they're equal using `.equals()` — then explain in a comment why `==` would be unreliable here.
- 5 Take a person's age and citizenship status (boolean) as input, and determine voting eligibility using `&&`, printing a proper message for each case.

Hard (5)

- 1 Build a simple calculator: take two numbers and an operator (+, -, *, /) as input using switch, and print the result (handle division by zero and invalid operator in default).
- 2 Given a date (day, month, year), validate if it's a real calendar date (handle Feb 29 leap year case, and months with 30 vs 31 days) using nested if-else — no libraries.
- 3 Simulate a basic ATM: take balance and withdrawal amount, handle cases — insufficient balance, invalid amount (negative/zero), and successful withdrawal, all with if-else only, using short-circuit checks to validate amount before checking balance.
- 4 Take a temperature and scale (C/F/K) as input using switch, convert it into the other two scales with correct formulas, and validate the scale input itself in the default case.
- 5 Write a program that takes 4 numbers and determines the relationship type: strictly increasing, decreasing, all equal, or unordered — using nested if-else only, without arrays or loops.

How to use this: Read Part 1 fully first — especially short-circuit evaluation and switch, since both show up directly in Part 2. Then solve questions in order. Once Hard feels doable, you're ready for the next topic (Loops).